



Automatic Exam Answer Sheet Grader

Under supervision of Eng. Rania

Khaled Ayman Farouk Ahmed
Mohamed Saber Labib Mostafa
Khaled Abdelnaser Mohamed Hassnain
Abdelrahman Housam Gamal El-Den Ahmad
Abdelfattah Elsayed Abdelfattah Mohammed

1. Project Overview

This project implements a fully automated pipeline for grading handwritten exam answer sheets. A scanned (or photographed) sheet is fed into the system, and the pipeline:

- Enhances the image using signal-processing algorithms written entirely from scratch with NumPy
- Extracts text from the processed image with a pre-trained OCR engine (RapidOCR).
- Parses the extracted text to reconstruct per-question answers.
- Compares those answers against a teacher-supplied answer key using multi-layer fuzzy matching.
- Outputs a detailed grade report to the console.

2. Architecture & Module Map

The project is structured as four collaborating Python modules:

Module	Responsibility
scratch_image.py	Pure-NumPy image processing library. Contains every algorithm used in the enhancement pipeline.
ocr_engine.py	Thin wrapper around RapidOCR. Loads the model once, resizes images when necessary, and returns bounding-box/text/confidence triples.
grader.py	Answer-key parser, multi-strategy fuzzy matcher, grade calculator, and console reporter.
main.py	Orchestrator. Calls the enhancement pipeline, invokes the OCR engine, groups OCR results into rows, assigns answers to questions, and triggers grading.

3. Image Enhancement Pipeline (scratch_image.py)

The enhancement pipeline converts a raw, possibly skewed, unevenly lit photograph of an answer sheet into a clean binary image suitable for OCR. The pipeline is executed in six sequential stages.

3.1 Stage 1 – Load & Grayscale Conversion

`load_grayscale(path)` Function:

The image is opened with Pillow, converted to the RGB colour space, and then collapsed to a single luminance channel using the ITU-R BT.601 luma coefficients:

```
gray = 0.299 * R + 0.587 * G + 0.114 * B
```

These weights reflect human perceptual sensitivity: the eye is most sensitive to green, moderately to red, and least to blue. Using float32 arithmetic before clipping to uint8 prevents integer overflow artefacts.

Input	Any colour image (JPEG / PNG / BMP ...)
Output	Single-channel uint8 NumPy array, values 0-255
Why	All subsequent algorithms operate on scalar pixel intensities; colour information is irrelevant for text.

3.2 Stage 2 – Skew Detection & Deskewing

`estimate_skew_angle()`, `rotate_image()`, `bilinear_sample()` Functions:

Scanned sheets are rarely perfectly aligned. Even a 1–2° tilt degrades OCR accuracy significantly because text-line projections become blurred. The deskew stage:

- Downscales the image to ≤ 450 px on the longest side for speed.
- Binarises with Otsu's global threshold (implemented from scratch).
- Sweeps candidate angles in $[-10^\circ, +10^\circ]$ with 0.5° steps.
- For each candidate angle, rotates the binary image and computes the variance of the first differences of the horizontal projection profile. Sharp text lines produce high variance; blurred lines produce low variance.
- Selects the angle with maximum projection-variance as the estimated skew.
- Applies a full-resolution bilinear rotation by the negative of that angle to straighten the image.

Iterates all 256 grey levels and maximises the between-class variance $\omega_{bg} \cdot \omega_{fg} \cdot (\mu_{bg} - \mu_{fg})^2$.

Otsu's Threshold :

Computes the inverse mapping for every destination pixel, clips source coordinates to the image boundary, and interpolates using four nearest-neighbour values with area-weighted blending.

Out-of-bounds pixels are filled with the median border intensity.

3.3 Stage 3 – Noise Removal (Median + Gaussian)

`median_filter()`, `gaussian_blur()`, `convolve()` Functions:

Two complementary filters are chained to suppress noise:

For every 3×3 neighbourhood, the centre pixel is replaced by the median of the 9 values. The median is inherently robust to impulse (salt-and-pepper) noise because outlier values never affect the median. Implemented with NumPy's `sliding_window_view` for efficiency, then `np.median`.

Median Filter (size=3):

Convolves with the separable 3×3 kernel. This attenuates high-frequency noise while preserving broad stroke geometry. Edge padding uses 'edge' mode to avoid border darkening.

Filter	Purpose
Median	Removes isolated speckles and scanner dust without blurring edges
Gaussian	Smooths residual gradient noise before thresholding

3.4 Stage 4 – Background Lighting Normalisation

`background_normalize()`, `box_mean()` Functions:

Uneven illumination (shadows, vignetting, folded paper) causes the background to vary across the sheet. A global threshold would then fail in shadowed regions. `background_normalize` corrects this:

- Computes a large-radius (`block_size=75`) box-mean filter using a 2-D integral image (summed-area table) for O(1) per-pixel cost. This produces a smooth estimate of the local background luminance.
- Computes `ink_response = max(background - image, 0)`, which is large where the pixel is darker than its local background (i.e., ink) and zero elsewhere.
- Scales the ink response: `output = 255 - ink_response × strength` (`strength=3.5`). Ink pixels become dark; background becomes 255.

```
background = box_mean(img, block_size=75)
ink_response = max(background - img, 0)
output = clip(255 - ink_response * 3.5, 0, 255)
```

The integral-image acceleration in `box_mean()` replaces $O(k^2)$ sliding-window summation with four array additions, making it viable for large block sizes without OpenCV.

3.5 Stage 5 – Adaptive Thresholding

`adaptive_text_threshold(block_size=51, offset=40)` Function:

After background normalisation the image is close to a clean grayscale, but residual gradients remain. A global threshold still risks cutting off strokes in bright regions. Adaptive thresholding computes a per-pixel decision:

```
local_mean = box_mean(img, block_size=51)
binary[y,x] = 255 if img[y,x] < local_mean[y,x] - 40
               = 0  otherwise
```

- `block_size=51` is large enough to span inter-word gaps but small enough to adapt to slow illumination changes.
 - `offset=40` prevents noise from being classified as ink; only pixels significantly darker than their neighbourhood are marked.
 - Output: binary image where 255 = ink, 0 = background.
-

3.6 Stage 5b – Small-Component Removal

`remove_small_components(min_area=80)` Function:

The thresholded binary image contains many small isolated white blobs (punctuation noise, scanner specks). A BFS (Breadth-First Search) connected-components labeller implemented from scratch visits every white pixel and floods its 8-connected neighbourhood. Any component whose pixel count is below `min_area` is discarded.

```
for every unvisited white pixel (sy, sx):
    flood-fill via BFS → collect pixels list
    if len(pixels) >= min_area:
        copy pixels to output
```

`min_area=80` is chosen empirically: a single handwritten letter typically exceeds 80 pixels, while noise blobs do not. The 8-connectivity (including diagonals) ensures that thin strokes are not accidentally split.

A separate variant `remove_border_components()` additionally discards small blobs near the image margins, removing staple holes and edge artefacts.

3.7 Stage 6 – Morphological Cleanup & Final Output

`clean_document_output()`, `close_image()`, `open_image()`, `erode()`, `dilate()`, `invert_binary()`

Functions:

The final stage polishes the binary image for human display while a separate OCR-safe image (Stage 5b + inversion) is routed to the OCR engine.

Fills small gaps and breaks within letter strokes caused by light ink or low contrast. `kernel_size=2`, `iterations = 1`.

Morphological Close (`dilate` → `erode`):

Removes residual thin bridges between adjacent characters. `kernel_size=2`, `iterations=1`.

Morphological Open (`erode` → `dilate`):

Final pass to clean margin noise (`min_area=1500` near borders).

File	Content
output/1_gray.jpg	Stage 1 grayscale
output/2_deskew.jpg	Stage 2 deskewed
output/3_denoise.jpg	Stage 3 denoised
output/4_clahe.jpg	Stage 4 background-normalised
output/5_threshold.jpg	Stage 5 binary
output/5b_ocr_safe.jpg	Stage 5b + inverted → sent to OCR
output/6_final.jpg	Stage 6 extra-clean display image

4. OCR Engine (ocr_engine.py)

The OCR layer is intentionally thin: it delegates all neural-network inference to RapidOCR (ONNX Runtime backend) and focuses on integration concerns such as lazy initialisation, image resizing, and confidence filtering.

4.1 Engine Initialisation

`get_reader()` uses the module-level singleton pattern (`_reader = None`) to ensure the ONNX model is loaded exactly once per process. Loading the model (~30-60 MB) is expensive; reusing it across calls avoids repeated disk I/O and GPU/CPU memory allocation.

```
def get_reader():
    global _reader
    if _reader is None:
        _reader = RapidOCR()
    return _reader
```

4.2 Image Preprocessing for OCR

Before inference the grayscale image is checked for excessive width:

```
if w > 1800:
    scale = 1800 / w
    img = resize_bilinear(img, scale)
```

The 1800-pixel cap balances recognition accuracy (character details must be visible) against inference latency (larger images are slower). `resize_bilinear()` (from `scratch_image.py`) is used for consistent quality with the rest of the pipeline.

4.3 Text Detection & Confidence Filtering

RapidOCR returns a list of (bounding_box, text, confidence) tuples. Each bounding box is a list of four [x, y] corner points in reading order (top-left → top-right → bottom-right → bottom-left).

Only detections with confidence ≥ 0.35 are retained. This threshold was tuned to reject garbled partial recognitions while preserving low-contrast but legible handwriting.

4.4 Result Format

Field	Description
bbox	List of 4 [x,y] points defining the word/line bounding box
text	Raw Unicode string recognised by the model
conf	Float in [0, 1]; model confidence in the recognition

5. Grading Logic (grader.py)

5.1 Answer-Key Parsing

`parse_answer_key(file_path)` Function:

The answer key is a plain-text file with one entry per line. Three formats are supported:

```
Q1 : mario          # colon with spaces
Q2: sekiro          # colon without spaces
Q3 GTA5             # space separator
```

The parser splits on ':' when present, otherwise on the first whitespace. It extracts the question number by filtering digit characters from the label (so 'Q1', 'q1', and '1' all map to 1). The result is a dict {int: str}.

5.2 OCR Text Normalisation

`normalize_ocr_text(text)` Function:

Before any comparison, both the recognised text and the correct answer are independently normalised:

- Lowercased.
- All non-alphanumeric characters stripped (regex `[^a-z0-9]`).
- All spaces removed (handles 'Cair 0' → 'Cair0').
- Pipe '|' → '1' (handled before stripping).

This aggressive normalisation intentionally loses punctuation and capitalisation information, which OCR is unreliable about for handwriting.

5.3 Character-Confusion Matching

`strings_match_with_ocr_confusion(s1, s2)` Function:

Even after normalisation, OCR makes systematic confusions between visually similar characters. The matcher checks whether two equal-length strings differ only in characters that belong to the same confusion group:

Confusion	Explanation
0 ↔ o	Zero vs letter O
1 ↔ l ↔ i	One vs lowercase L vs uppercase I
5 ↔ s	Five vs letter S
8 ↔ b	Eight vs letter B
2 ↔ z	Two vs letter Z

6 ↔ g ↔ 9	Six vs G vs Nine
9 ↔ q ↔ g	Nine vs Q vs G

5.4 Levenshtein (Edit-Distance) Matching

levenshtein_distance(s1, s2) Function:

For answers of length ≥ 4 , a single-row dynamic-programming implementation of the Levenshtein edit distance is computed. If the distance is ≤ 1 (one insertion, deletion, or substitution), the answer is accepted. This handles isolated OCR errors such as a dropped character.

```
# Rolling single-row DP
previous = list(range(len(s2) + 1))
for i, c1 in enumerate(s1, start=1):
    current = [i]
    for j, c2 in enumerate(s2, start=1):
        current.append(min(
            current[j-1] + 1,      # insert
            previous[j] + 1,       # delete
            previous[j-1] + (c1!=c2) # replace
        ))
    previous = current
return previous[-1]
```

5.5 Fuzzy Match Pipeline

fuzzy_match(recognized, correct, case_sensitive=False) Function:

The four matching strategies are applied in order of strictness; the first match short-circuits the chain:

Strategy	Condition
1. Exact	Lowercased raw strings are identical
2. Normalized exact	After normalize_ocr_text both strings match
3. Confusion match	strings_match_with_ocr_confusion returns True
4. Edit distance	Levenshtein ≤ 1 (strings ≥ 4 chars)
5. Containment	Normalised correct is a substring of normalised recognised (or vice versa, ≥ 3 chars)

5.6 Grade Computation

grade_answers(recognized_answers, correct_answers) Function:

Iterates over every question number in the answer key (sorted). For each question, calls `fuzzy_match`. Accumulates `correct_count`. Returns a dict containing `total_questions`, `correct_count`, `score_percent`, and a per-question details list.

```
score_percent = round(correct_count / total * 100, 1)
```

5.7 Grade Report Output

print_grade_report(result) Function:

Prints a formatted console report showing CORRECT / WRONG status, the expected answer, the recognised answer, and (for wrong answers) the normalised forms of both sides to aid debugging.

6. Main Orchestration (main.py)

main.py ties all modules together and adds the non-trivial step of parsing raw OCR detections into structured question–answer pairs.

6.1 OCR Result Parsing & Row Grouping

`group_detections_into_rows()`, `compute_adaptive_threshold()` Functions:

OCR engines return detections in arbitrary order. Text on the same line must be merged before a question label can be associated with its answer.

- `compute_adaptive_threshold()` estimates an appropriate vertical clustering distance by sorting all bbox centre-Y coordinates, computing inter-centre gaps, and taking half the median gap.
- `group_detections_into_rows()` sorts detections by Y, then scans linearly: detections within `y_threshold` of the current row's Y are merged; a larger gap starts a new row.
- Within each row, detections are sorted by X (left → right).

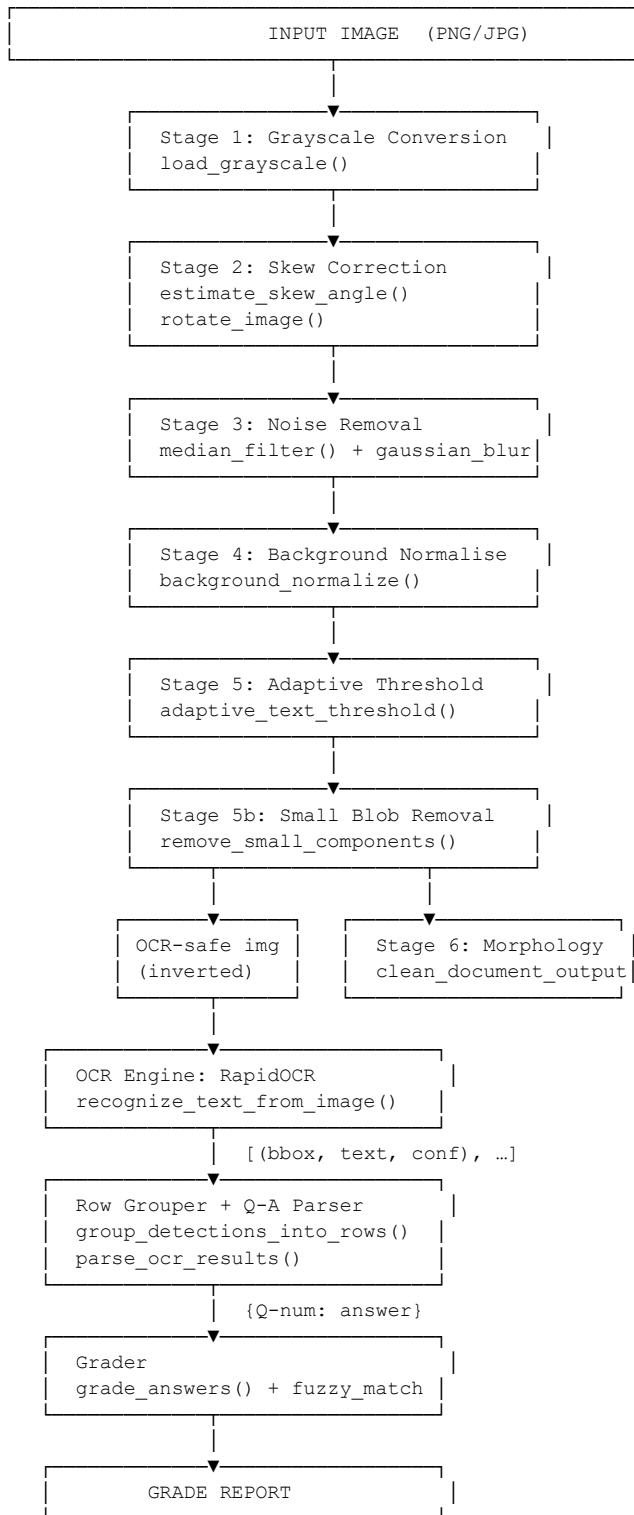
6.2 Question–Answer Assignment

`parse_ocr_results(results, num_questions)` Function:

A three-pass strategy assigns text to questions:

Pass	Logic
Pass 1 – Label match	Regex <code>r'[QqKkR]\s*(\d+)</code> searches each row. If found, the text after the label becomes the answer for that question number.
Pass 2 – Subscript correction	Some OCR models read 'Q5' as 'Qs' (subscript 5 → s). A mapping <code>SUBSCRIPT_MAP</code> converts common OCR letter substitutions back to digits.
Pass 3 – Numeric label	Rows that failed both passes are re-examined: if they start with a digit followed by text, the digit is treated as the question number.
Pass 4 – Unassigned fallback	Any remaining text rows are matched sequentially to still-missing question numbers.

7. End-to-End Data-Flow Diagram

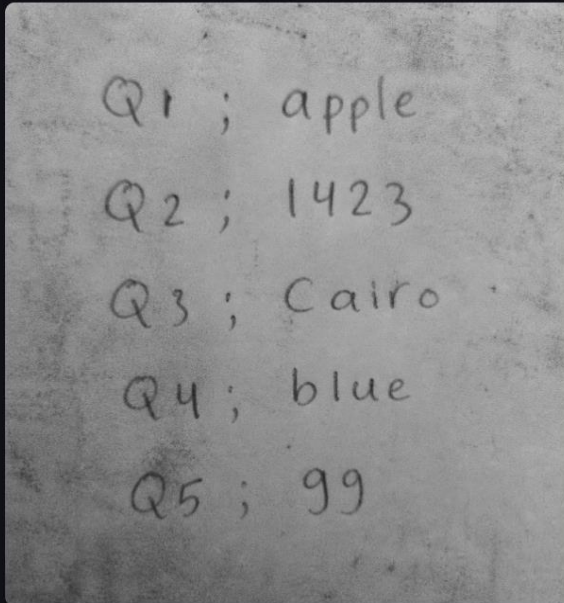


10. Output Sample

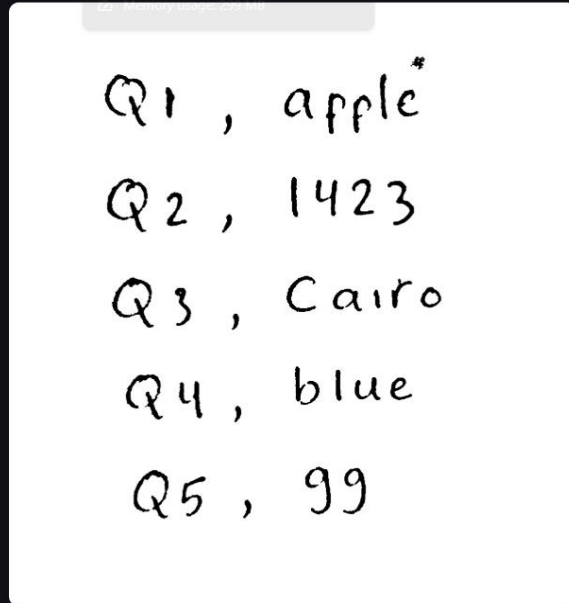


Image Enhancement

ORIGINAL — NOISY INPUT

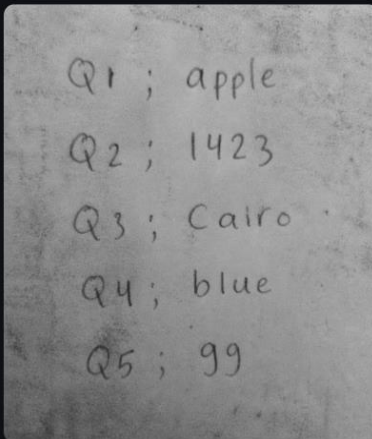


ENHANCED — CLEAN OUTPUT

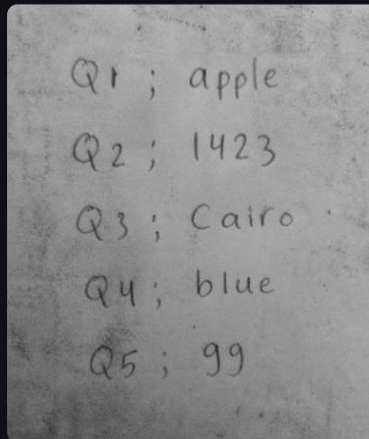


Preprocessing Stages

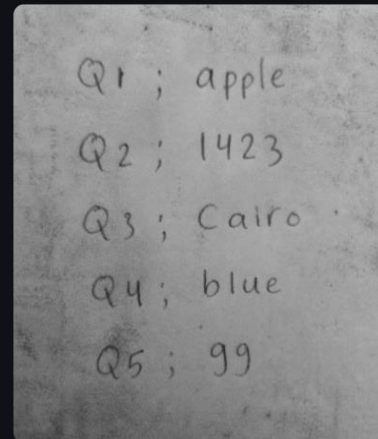
STAGE 1 - GRAYSCALE INPUT

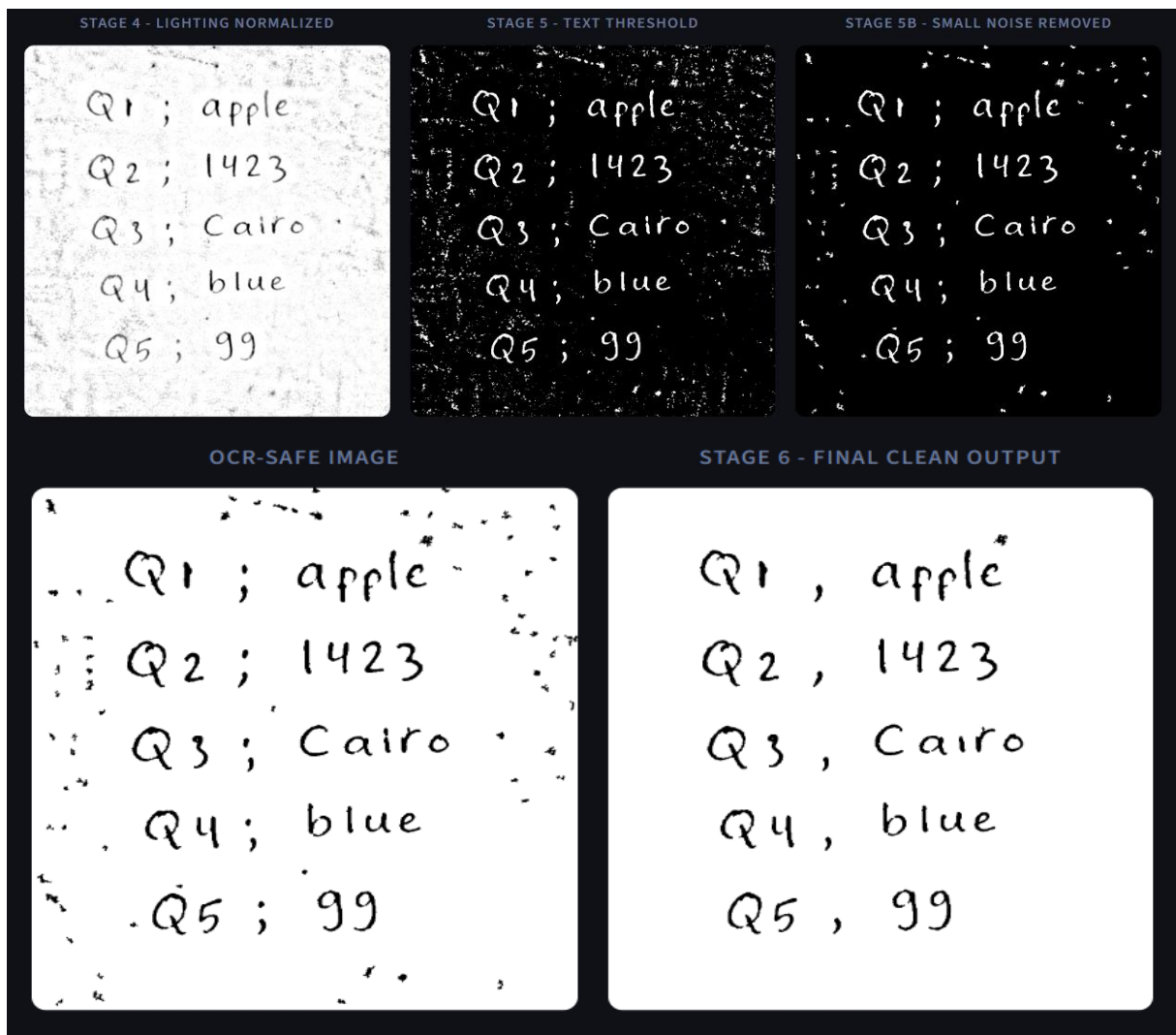


STAGE 2 - DESKEW



STAGE 3 - DENOISE





Question Breakdown				
Q#	RECOGNIZED	CORRECT ANSWER	MATCH	RESULT
Q1	apple normalized: apple	apple	—	✓ Correct
Q2	1423 normalized: 1423	1423	—	✓ Correct
Q3	Cairo normalized: cairo	Cairo	—	✓ Correct
Q4	blue normalized: blue	blue	—	✓ Correct
Q5	99 normalized: 99	99	—	✓ Correct

11. Conclusion

The Automatic Exam Answer Sheet Grader demonstrates that a production-quality document-analysis pipeline can be built with minimal external dependencies. The image enhancement layer — comprising grayscale conversion, skew correction, median and Gaussian denoising, integral-image background normalisation, adaptive thresholding, connected-component filtering, and morphological cleanup — is implemented entirely from first principles using NumPy.

The OCR layer delegates to RapidOCR for neural text detection and recognition, but wraps it with confidence filtering and resolution management. The grading layer applies a five-strategy fuzzy matching pipeline that is robust to the systematic character confusions inherent in handwriting OCR.

Together, the three layers form a cohesive, modular system that can be extended — for example, by adding support for multiple-choice bubble detection, multi-page sheets, or a web-based submission interface — without architectural changes to the core pipeline.

Possible Future Enhancements

- Perspective / homography correction for photos taken at an angle.
- Deep-learning-based skew detection for finer angle precision.
- OMR (Optical Mark Recognition) for multiple-choice questions.
- Arabic / RTL handwriting support via dedicated OCR models.